

Analytical Adaptive Algorithm for Autonomous Race Driving

Neil Sambhu¹ and Srinivas Katkoori²

Abstract—Autonomous race driving is a difficult problem, as the driving algorithm needs to find unique maneuvers on the track to achieve the fastest lap time. To date and to the best of our knowledge, there exists no algorithm that supersedes human performance. In this work, we propose an analytical self-driving algorithm for race driving. The self-driving algorithm will use a baseline (i.e., generated *a priori* by manually driving on the track) for course correction while attempting to achieve the shortest lap time. The proposed algorithm iteratively determines the steer angle, throttle, and brake controls while adhering as close to the baseline, by computing deviations between (1) the predicted location at the next time step and (2) the baseline location closest to predicted. We report results for various fixed speeds to demonstrate the correctness of the algorithm. We then demonstrate more realistic adaptive driving that greedily increases the speed whenever the steering correction is small. The proposed algorithm is implemented and validated in the open-source CARLA Simulator v0.9.14. Experimental results for three different car models demonstrate that the adaptive algorithm is effective in achieving good lap times.

I. INTRODUCTION AND RELATED WORK

Track driving is a difficult problem to solve because (1) the car needs to find the path to drive around the track; (2) the optimal path to drive the lap changes as the car begins to drive faster; and (3) optimization of the drive involves continuous values in time and space (i.e., discretizing the track space is a precondition for optimizing lap time).

A. Self-Driving Cars

Self-driving cars (SDCs) use deep learning [1] and convolutional neural networks (CNNs) [2]. Vision and sensing are important components within autonomous driving. Within reinforcement learning, Q-learning is an important aspect of self-driving cars [3]. Simulators are useful for helping to test autonomous driving strategies at minimal expense [4]. The Ackerman Steering Angle is a fundamental control to compute for a self-driving car [5]. To solve full autonomy, autonomous vehicles must operate in challenging road conditions [6]. After having solved the driving problem, full autonomy requires an understanding of vehicle navigation [7].

B. Related Work

Limitations exist in the literature of benchmarking autonomous, high-performance cars in simulation. There exists

content in the literature for reinforcement learning in self-driving cars [9]. Herman *et al.* use a platform that is no longer supported as of 2024.

Hermansdorfer *et al.* benchmarked Technical University of Munich (TUM) Roborace autonomous racing software against a professional human race driver in the real world [8]. The authors used 14 key performance indicators (KPIs) to quantify human driver and software performance. As such, the human driver is better than the software across all metrics. The software is novel; further research will allow software performance to meet or exceed human standards. There is an expectation machines learn from humans and beat them. As shown in Table I, TUM Roborace software cannot beat humans [8].

Wu *et al.* designed an autonomous racecar [10]. The authors use Vrep simulator. The software framework has perception, planning, and control. The perception uses camera and LiDAR. The camera-based perception uses YOLO v3 object detector. The LiDAR-based method clusters points to find objects. Path planning uses triangulation and penalty minimization. Control modifies the steering angle. The authors do not list quantifiable results; as such, the paper lacks metrics to evaluate performance.

Salem *et al.* designed car controllers for racing simulators [11]. The paper uses The Open Racing Car Simulator (TORCS). The simulator allows competing against computer-controlled opponents. There are (1) many sensors and (2) controllers and fuzzy sub-controllers. The genetic algorithm tries to find the optimal parameters for driving. For results, the $GFC - MMS_1$ is better at fast laps; the $GFC - AVS_2$ achieves the best score against other drivers. In their subsequent work, Salem *et al.* used new optimizations for their controllers [12]. The $GFC - VA$ is best. For limitations, the controllers from both papers need real-world validation.

Liu *et al.* used deep reinforcement learning in simulation for autonomous racing [13]. The authors used The Open Racing Car Simulator (TORCS). The paper represents the state of the vehicle. The proposed architectures are as follows: (1) dense layers, (2) Recurrent Neural Network (RNN), and (3) Long Short-Term Memory (LSTM). Results show the LSTM is usually best. The methods need real-world validation.

Trumpp *et al.* used residual policy learning (RPL) to control autonomous racing cars in simulation [14]. The paper uses the F1TENTH simulator. The authors use minimum curvature trajectory planning to generate racing lines. There is a base and a residual controller. The authors use reinforcement learning as a larger part of a proximal policy optimization (PPO) network. The residual controller gives an average of

¹Neil Sambhu is a PhD Candidate in Computer Science, Computer Science and Engineering Department, University of South Florida, 4202 E. Fowler Avenue Tampa, FL 33620, USA nsambhu@mail.usf.edu

²Srinivas Katkoori is a Professor with the Department of Computer Science and Engineering, University of South Florida, 4202 E. Fowler Avenue Tampa, FL 33620, USA katkoori@mail.usf.edu

TABLE I: From the key performance indicators, human driving performance exceeds the capabilities of the autonomous racecar software (i.e., TUM Roborace) [8].

key performance indicator	human race driver	TUM Roborace software	relative difference to human
max. velocity in ms^{-1} (higher is better)	61.5	58.0	-6%
max./min. long. acceleration in ms^{-2} (higher is better)	5.0/-17.4	4.7/9.2	-6%/-47%
max. lat. acceleration in ms^{-2} (higher is better)	16.5	13.7	-17%
lap time in s (lower is better)	63.03	69.98	+11%
coasting time in s (lower is better)	1.99	3.65	+83%
steering speed in $rads^{-1}$ (higher is better)	1.39×10^{-4}	6.94×10^{-5}	-50%

a 4.55% improvement over the standard controller. Regarding limitations, the FITENTH uses 1/10th scale cars with scaled-down road courses; as such, the lap times (e.g., Nürburgring) are not reflective of published lap times for real cars.

Table II summarizes the literature review and compares the prior work with this work. This is a baseline work that will be improved; no prior work readily exists that we can extend in CARLA Simulator to implement the algorithm.

II. PROPOSED METHOD

Algorithm 1 Steer controller logic

```

1: Input:  $l_{current} \leftarrow$  current location as x-y pair
2: Input:  $l_{prediction} \leftarrow$  extrapolated location at next time-step, assuming vehicle controls are constant
3: Input:  $l_{path} \leftarrow$  location from ground-truth path closest to  $l_{prediction}$ 
4: Input:  $throttle, steer, brake \leftarrow$  current vehicle controls
5: Output:  $throttle, steer, brake \rightarrow$  next vehicle controls
6:
7: Subroutine 1: create vectors from input locations
8:  $\vec{v}_1 \leftarrow -l_{current} + l_{prediction}$ 
9:  $\vec{v}_2 \leftarrow -l_{current} + l_{path}$ 
10:
11: Subroutine 2: compute direction of deviation from ground-truth path
12: Declaration:  $d_{deviation}$  represents the direction of deviation from ground-truth path (i.e., -1 is left, 0 is straight, 1 is right)
13:  $d_{deviation} \leftarrow sign(\vec{v}_1 \times \vec{v}_2)$ 
14:
15: Subroutine 3: compute steering angle
16:  $\Delta\theta_{steer}, steer_{max} \leftarrow STEERANGLE()$ 
17:
18: Subroutine 4: compute steer signal value
19:  $steer \leftarrow STEERSIGNAL(\Delta\theta_{steer}, steer_{max}, steer)$ 
20:
21: Subroutine 5: compute throttle and brake signal values
22:  $throttle, brake \leftarrow TB(s, throttle, brake)$ 
23:
24: Subroutine 6: potentially no steering correction
25: if  $\Delta\theta_{steer} \leq 5$  and boolFastDrive then
26:    $throttle, steer, brake = 0.75, 0.0, 0.0$ 
27: end if
28:
29: Subroutine 7: output vehicle control signals
30: return  $throttle, steer, brake$ 

```

Algorithm 2 Steer Angle

```

1: function STEERANGLE( )
2:    $\|\vec{v}_{min}\| \leftarrow 5$  as minimum speed requirement in km/h
3:    $steerDivisor \leftarrow 50$  as steering scalar
4:    $s \leftarrow$  magnitude of vehicle velocity in km/h
5:   Declaration:  $s_{max}$  represents maximum steering
6:    $\Delta\theta_{steer} \leftarrow -1 * d_{deviation} * \arccos(\frac{\vec{v}_1 \cdot \vec{v}_2}{\|\vec{v}_1\| \|\vec{v}_2\|})$ 
7:   if  $s < \|\vec{v}_{min}\|$  then
8:      $steer_{max} \leftarrow 0.01$ 
9:   else
10:     $steer_{max} \leftarrow \min(\frac{|\Delta\theta_{steer}|}{steerDivisor}, 1)$ 
11:   end if
12:   return  $\Delta\theta_{steer}, steer_{max}$ 
13: end function

```

Algorithm 3 Steer Signal

```

1: function STEERSIGNAL( $\Delta\theta_{steer}, steer_{max}, steer$ )
2:    $\Delta unitSteer \leftarrow 0.1$ 
3:   if  $\Delta\theta_{steer} > 0$  then
4:      $steer \leftarrow \max(steer - \Delta unitSteer, -steer_{max})$ 
5:   else
6:      $steer \leftarrow \min(steer + \Delta unitSteer, steer_{max})$ 
7:   end if
8:   return  $steer$ 
9: end function

```

Algorithm 4 Throttle and brake logic

```

1: function TB( $s, throttle, brake$ )
2:    $\|\vec{v}_{target}\| \leftarrow 30$  as target speed in km/h
3:    $\Delta unitThrottle \leftarrow 0.1$ 
4:    $\Delta unitBrake \leftarrow 0.1$ 
5:   if  $s < \|\vec{v}_{target}\|$  then
6:      $brake = 0.0$ 
7:      $throttle = \min(throttle + \Delta unitThrottle, 1.0)$ 
8:   else
9:      $throttle = 0.0$ 
10:     $brake = \min(brake + \Delta unitBrake, 1.0)$ 
11:   end if
12:   return  $throttle, brake$ 
13: end function

```

TABLE II: Related Work in Autonomous Racecars

Paper	Simulator	Method	Remarks
Hermansdorfer <i>et al.</i> [8]	None	benchmark software against human	Results: human is better at all key performance indicators. Limitations: N/A (software still needs to reach parity)
Wu <i>et al.</i> [10]	Vrep	vehicle triangulation	Results: simulated driving without any quantifiable results. Limitations: lack of metrics to evaluate performance.
Salem <i>et al.</i> [11], [12]	TORCS	evolutionary algorithm (i.e., best among various parameters)	Results: GFC-MMS1 is better at fast laps. GFC-AVS2 achieves the best score against other drivers [11]. GFC-VA controller is best [12]. Limitations: need real-world validation
Liu <i>et al.</i> [13]	TORCS	dense layers, recurrent neural network (RNN), long short-term memory (LSTM)	Results: LSTM is usually best Limitations: need real-world validation
Trumpf <i>et al.</i> [14]	F1TENTH	reinforcement learning (RL): proximal policy optimization (PPO)	Results: residual controller gives an average of 4.55% improvement over standard controller. Limitations: The Nürburgring lap time is not realistic (i.e., F1TENTH cars and tracks are 1/10th scale.)
Sambhu <i>et al.</i> (this paper)	CARLA	triangulated steering control navigating to closest point on path	Results: less than 1.6-meter deviations from ground-truth path from target speeds of 30 to 80 km/h. Limitations: vehicle guidance is toward established path and not constrained to width of track.



Fig. 1: Above is the aerial view of CARLA Simulator Town 6. The path in this paper follows the outer perimeter of the map, starting from the bottom-left corner of the map, passing through the subsequent three corners in a counterclockwise fashion to end at the bottom-left corner.

A. Proposed Approach

We have implemented the proposed algorithm in CARLA Simulator 0.9.14 [15]. CARLA Simulator is an open-source driving simulator designed for simulating autonomous driving in urban environments. CARLA Simulator supports an array of sensors and multiple environmental conditions.

The steering controller started from the automatic control script of the CARLA Python API. The current implementation makes use of the command-line arguments, connection to the simulator, vehicle spawning, and camera.

The script uses a text file of three-dimensional locations as input to create the ground-truth drive. The function *getLocationClosestToCurrent()* takes as input a single location and outputs a pair of (1) the distance to the closest

point and (2) the closest point. As shown in Figure 1, the ground-truth path in this paper uses Town 6.

To use Algorithm 1, the user drives around the track manually to collect locations. These locations become the ground-truth path for the algorithm.

Assumptions for the algorithm to work are as follows: (1) precise location of GPS, (2) sensors are accurate for computing velocity, and (3) there is a real-time computer on board to compute the output for the vehicle controls (i.e., steer, brake, and throttle).

As shown in Figure 4, the steering controller usually operates within bounds of 5 degrees of correction from the ground-truth path. From Figure 5, the vehicle closely follows the ground-truth path.

This approach omits machine learning and instead uses classical AI as trigonometry to implement the steering control of the vehicle.

B. Algorithm

As shown in Algorithm 1 and Figure 2, the steering controller logic takes as input (1) three location values and (2) three scalars representing the current vehicle controls. Algorithm 1 outputs the vehicle controls for the next time-step.

1) *Inputs and Outputs:* The locations are x-y pairs representing (1) the current vehicle location; (2) the extrapolated vehicle location at the next time-step, assuming vehicle controls are constant; and (3) the location from the ground-truth path closest to the predicted vehicle location at the next time-step. The vehicle controls are scalar values of throttle $\in [0, 1]$, steer $\in [-1, 1]$, and brake $\in [0, 1]$. The output has the same format as the input: throttle, steer, and brake values for the next time-step.

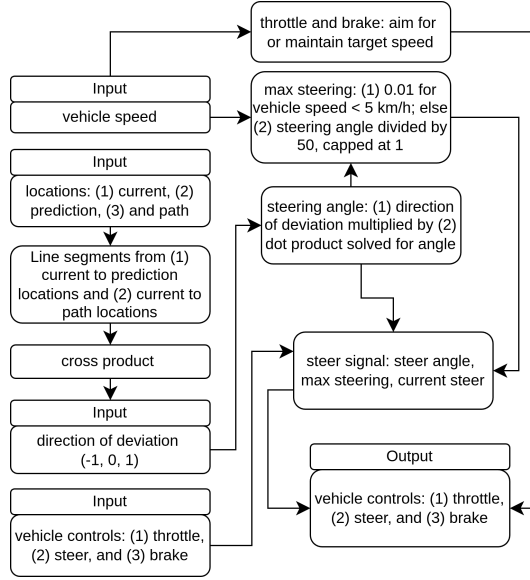


Fig. 2: The flowchart shows the process flow for computing the throttle, steer, and brake values for the next time-step. This diagram is simplified and omits the adaptive fast-drive method (i.e., when steer signal is less than or equal to five degrees, (1) throttle is 0.75 and (2) steer and brake are zero).

2) *Direction of Deviation:* The first component of Algorithm 1 computes the direction of deviation from the ground-truth path. The current, predicted, and path locations are assigned to a , b , and c , respectively. The direction of deviation is the cross product of (1) the line segment from the current to predicted locations with (2) the line segment from the current to path locations. Algorithm 1 discretizes the deviation by sign (i.e., set deviation to -1 for negative values; set 1 for positive values).

3) *Maximum Steering:* The second component of Algorithm 1 computes the maximum steering value. The minimum speed requirement is 5 km/h. Algorithm 1 gets the vehicle speed. The algorithm uses the dot product to compute the angle between the line segments from the current location to (1) the predicted and (2) path locations. To account for the angle the car needs to steer, Algorithm 1 multiplies the angle between the two line segments by the opposite sign of the direction of deviation. If the vehicle speed is less than the minimum speed, the maximum steering value is set to 0.01; otherwise, the maximum steering value is set to the minimum of (1) the absolute value of the angle between the two line segments divided by 50 and (2) the scalar 1.

4) *Target Speed:* The target speed is 30 km/h. There are two variables of 0.1 to act as units to change the throttle and brake values. For each time-step, if the vehicle speed is less than the target speed, the brake is set to 0 and the throttle is increased by 0.1, capped at 1; otherwise, the throttle is set to 0 and the brake is increased by 0.1, capped at 1.

5) *Output Values:* Algorithm 1 outputs the throttle, steer, and brake values for the next time-step.

6) *Complexity Analysis:* Algorithm 1 has 3 location inputs (i.e., x - y pairs) and 3 vehicle control signal inputs (i.e.,

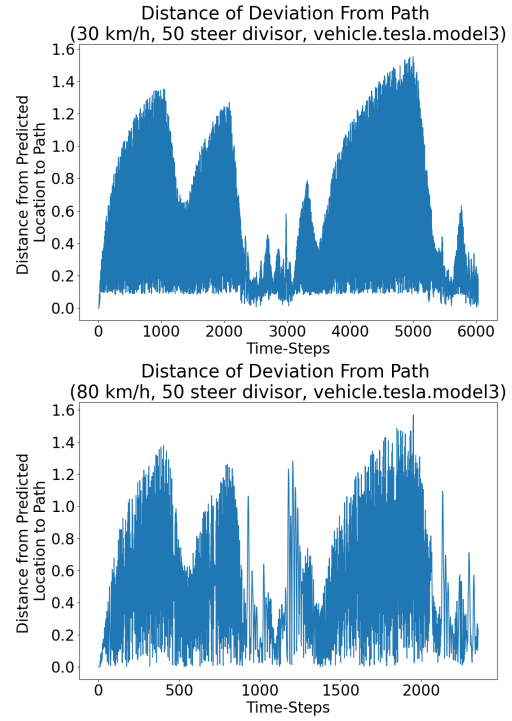


Fig. 3: The graphs show the distances from (1) the predicted location at the next time-step in the future and (2) the closest location from the ground-truth path at (3) target speeds of 30 and 80 km/h. Each time-step is 1/20th of a second.

scalars). The outputs are 3 vehicle control signals. The time complexity is $O(1)$; the space complexity is $O(1)$. For compute before running Algorithm 1, finding the location from the ground-truth path closest to the predicted vehicle location at the next time-step (i.e., $l_{prediction}$) has time complexity $O(n)$ for n locations in the path.

C. Adaptive Fast-Drive Method

The adaptive fast-drive method is as follows: lap times exist where the vehicle stays on three-quarters throttle when the steering correction is less than or equal to five degrees; steer value is set to straight. When the steering correction is greater than five degrees, the vehicle modulates throttle, aiming for $\|\vec{v}_{target}\|$.

The code is available in the following repository: <https://github.com/neilsambhu/carla-race/tree/2024.05.ITSC>.

III. EXPERIMENTAL RESULTS

Results show path following has close adherence to the ground-truth path. The distance between two lane lines is 3.5 meters. As shown in Figure 3, the distance of deviation from the ground-truth path is within 1.5 meters. The ground-truth drive takes advantage of the road course being 5 lanes wide by driving across the full width of the road through turns. At faster speeds, there is less distance of deviation. Similarly, as shown in Figure 4, the steering correction necessary to keep the car on the ground-truth path reduces as vehicle speed

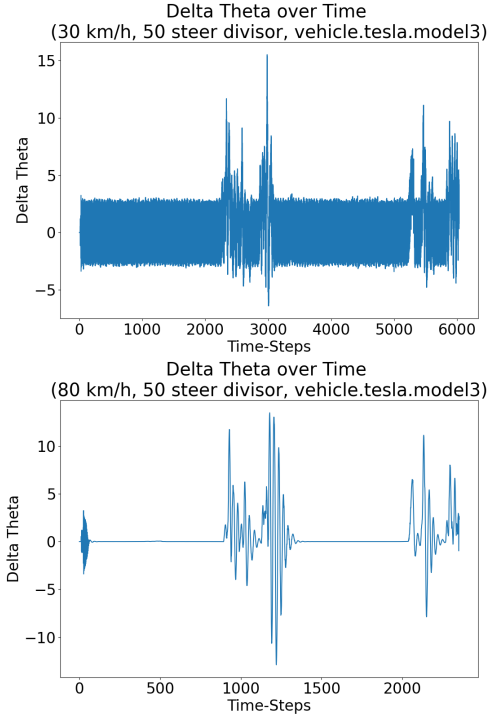


Fig. 4: The graphs show (1) the computation of the computed corrective steering angle needed to maintain the vehicle trajectory on the path of the ground-truth drive (2) with respect to time at (3) target speeds of 30 and 80 km/h.

TABLE III: Baseline Lap Times

Target Speed (km/h)	Lap Time (s)
30	302.10
40	228.85
50	184.25
60	154.70
70	133.60
80	118.10

The table shows the lap times at target speeds of 30 through 80 km/h, at 10 km/h increments. In Algorithm 4 line 2, the scalar $\|\vec{v}_{target}\|$ changes. In Algorithm 2 line 3, *steerDivisor* is held constant at 50. The vehicle is a Tesla Model 3.

increases. From Figure 5, the vehicle closely follows the ground-truth path.

Experimentation starts with *steerDivisor* = 50. As shown in Table III, the path-following algorithm works well at maintaining speeds from 30 km/h to 80 km/h. At a target speed of 90 km/h, the vehicle loses control and is not able to finish the drive.

Below 30 km/h, the vehicle is unable to follow the path at 20 km/h and instead drives in a circle; presumably, even less steering correction is needed to maintain path following at speeds below 30 km/h.

From Table IV, setting the target speed to 30 km/h, the lap time is similar across 4 steer divisor values.

Table V also demonstrates that at target speed 80 km/h also the steer divisor does not affect lap time.

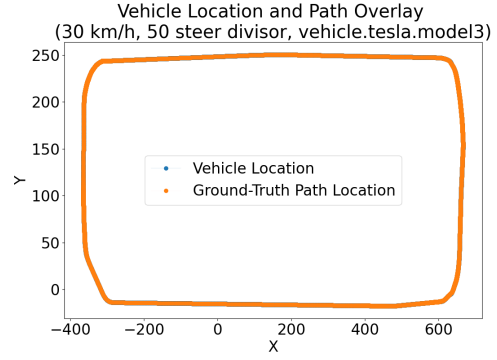


Fig. 5: The graph shows the (1) vehicle location travelled relative to the (2) path of the ground-truth drive. The ground-truth drive is from a manual drive around the course. The graph is consistent across target vehicle speeds from 30 to 80 km/h.

TABLE IV: Lap Times with Changing Steer Divisor (30 km/h) for the Tesla Model 3

Steer Divisor	Lap Time (s)
25	302.50
50	302.10
75	302.25
100	302.35

As shown in Table VI, trying different vehicles (i.e., Tesla Model 3, Ford Ambulance, and Ford Crown (taxi)) showcases the difference in performance of this steering control algorithm. This steering control algorithm works best on the Ford Ambulance at 111.75 seconds, a 5.38% improvement from the baseline Tesla Model 3. The vehicles could not complete the course at speeds of 90 km/h and higher.

A. Adaptive Fast-Drive Method

The Adaptive Fast-Drive Method still does not use machine learning. From Table VII, the adaptive fast-drive method improves vehicle lap times.

As shown in Table VII, there are speeds too low and too high where the adaptive fast-drive method will not work. The Tesla Model 3 and Ford Ambulance had the fastest laps at target speeds of 70 km/h. The Ford Crown had the fastest

TABLE V: Lap Times with Changing Steer Divisor (80 km/h) for Tesla Model 3

Steer Divisor	Lap Time (s)
25	117.65
50	118.10
75	118.30
100	118.35

TABLE VI: Lap Times for Different Vehicles

Vehicle	Lap Time (s)
Tesla Model 3	118.10
Ford Ambulance	111.75
Ford Crown (taxi)	115.10

TABLE VII: Lap Times with Adaptive Fast-Drive Method

Vehicle	Target Speed (km/h)					
	30	40	50	60	70	80
Tesla Model 3	125.00	123.55	118.35	115.20	110.80	U-turn
Ford Ambulance	148.20	132.25	129.65	122.85	120.15	Crash
Ford Crown (taxi)	Speed dropped	U-turn	116.45	117.20	Speed dropped	Speed dropped

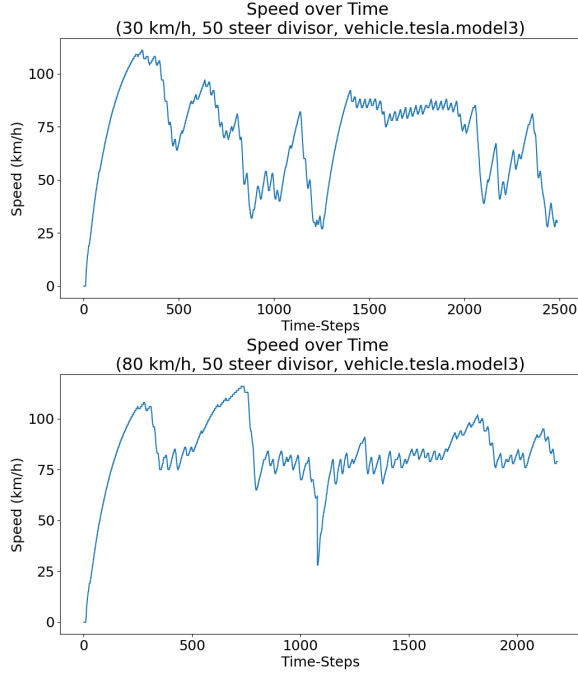


Fig. 6: Adaptive fast-drive method at 30 and 80 km/h for the Tesla Model 3.

lap at a target speed of 50 km/h. As shown in Figure 6, the Tesla Model 3 reaches a top speed of 100 km/h using the adaptive fast-drive method. When steering correction is less than or equal to 5 degrees, the vehicle applies three-quarters throttle with no steering correction. In Algorithm 2 line 3, the scalar *steerDivisor* is held constant at 50. In Algorithm 4 line 2, $\|\vec{v}_{target}\|$ changes in increments of 10 from 30 to 80. When a cell in the table does not include a lap time, there is a specified reason why the vehicle did not provide a time: (1) the speed dropped too low (i.e., fishtail, oversteer); (2) the vehicle made a U-turn to head back to the finish line instead of staying on route; or (3) the vehicle crashed.

At the minimum operable bounds, no vehicle could complete the road course with a target speed of 80 km/h. Additionally, the Ford Crown could not complete at target speeds of 30, 40, and 70 km/h. Future work will use machine learning to make this method more stable.

IV. CONCLUSION

The vehicle driving algorithm in this paper is a classical form of artificial intelligence (AI). Results are promising for establishing a baseline for fast vehicle driving around Town06 in CARLA Simulator. Further research is necessary to improve the baseline lap times. Future work will include reinforcement learning software, where the algorithm discretizes the lap to reward fast travel through the track.

Further, we can use the data from human drivers in both offline (i.e., dataset) and online (i.e., real time) settings.

REFERENCES

- [1] B. Asmika, G. Mounika, and P. S. Rani, "Deep learning for vision and decision making in self driving cars-challenges with ethical decision making," in *2021 International Conference on Intelligent Technologies (CONIT)*, pp. 1–5, 2021.
- [2] T.-D. Do, M.-T. Duong, Q.-V. Dang, and M.-H. Le, "Real-time self-driving car navigation using deep neural network," in *2018 4th International Conference on Green Technology and Sustainable Development (GTSD)*, pp. 7–12, 2018.
- [3] T. Okuyama, T. Gonsalves, and J. Upadhyay, "Autonomous driving system based on deep q learnig," in *2018 International Conference on Intelligent Autonomous Systems (ICoIAS)*, pp. 201–205, 2018.
- [4] M. Alsherif, M. Daowd, A. Bassiuny, and H. A. Metered, "Utilizing transfer learning in the udacity simulator to train a self-driving car for steering angle prediction," in *2023 Eleventh International Conference on Intelligent Computing and Information Systems (ICICIS)*, pp. 134–139, 2023.
- [5] Z. M. U. Din, W. Razzaq, U. Arif, W. Ahmad, and W. Muhammad, "Real time ackerman steering angle control for self-driving car autonomous navigation," in *2019 4th International Conference on Emerging Trends in Engineering, Sciences and Technology (ICEEST)*, pp. 1–4, 2019.
- [6] P. Mehune and S. Dorle, "Robust and adaptive neural networks for self-driving cars in challenging road conditions," in *2024 2nd International Conference on Computer, Communication and Control (IC4)*, pp. 1–5, 2024.
- [7] V. Dementiev, K. Sviatov, and M. Gaponova, "Image processing algorithms for self-driving cars navigation," in *2022 VIII International Conference on Information Technology and Nanotechnology (ITNT)*, pp. 01–04, 2022.
- [8] L. Hermansdorfer, J. Betz, and M. Lienkamp, "Benchmarking of a software stack for autonomous racing against a professional human race driver," in *2020 Fifteenth International Conference on Ecological Vehicles and Renewable Energies (EVER)*, pp. 1–8, 2020.
- [9] J. Herman, J. Francis, S. Ganju, B. Chen, A. Koul, A. Gupta, A. Skabelkin, I. Zhukov, M. Kumskey, and E. Nyberg, "Learn-to-race: A multimodal control environment for autonomous racing," in *proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 9793–9802, 2021.
- [10] S. Wu, Z. Yang, X. Xie, Y. Wang, X. Wang, Q. Wang, B. Wu, H. Zhang, H. Zhang, H. Ma, X. Zhang, and H. Lin, "Design and simulation of an autonomous racecar: Perception, slam, planning and control," in *2021 IEEE International Conference on Autonomous Systems (ICAS)*, pp. 1–5, 2021.
- [11] M. Salem, A. M. Mora, and J. J. Merelo, "The evolutionary race: Improving the process of evaluating car controllers in racing simulators," in *2018 IEEE Conference on Computational Intelligence and Games (CIG)*, pp. 1–8, 2018.
- [12] M. Salem, A. M. Mora, and J. J. Merelo, "Beating uncertainty in racing bot evolution through enhanced exploration and pole position selection," in *2019 IEEE Conference on Games (CoG)*, pp. 1–8, 2019.
- [13] R. Liu, W. Zhuang, F. Tong, and G. Yin, "Learn to race: Sequential actor-critic reinforcement learning for autonomous racing," in *2023 IEEE International Automated Vehicle Validation Conference (IAVVC)*, pp. 1–6, 2023.
- [14] R. Trumpp, D. Hoornaert, and M. Caccamo, "Residual policy learning for vehicle control of autonomous racing cars," in *2023 IEEE Intelligent Vehicles Symposium (IV)*, pp. 1–6, 2023.
- [15] S. Malik, M. A. Khan, and H. El-Sayed, "Carla: Car learning to act—an inside out," *Procedia Computer Science*, vol. 198, pp. 742–749, 2022.